

Software Development with AI

1

Neil Cherry, BS-CS

Joseph E. Wilkes, PhD-EE



September 2026



Welcome to an Adventure

2



Neil Cherry, BS-CS

3

- Retired, Network Systems Engineer/Quality Assurance Software Engineer
- Starting in Computers in '78, started using Emacs in '79, still use it
- Run the CDL (Computer Deconstruction Lab)
- Hobbies: Cycling, Computer Software and Hardware.
- ET & CS degrees
- Been using Linux since March, 1992
- Unix since 1984
- Wrote a book on Home Automation
- ncherry@linuxha.com



September 2026



Joseph E. Wilkes, PhD-EE

4

- Life Fellow of IEEE
- Started Programming in Fortran 1969-70
- UNIX - 1975
- Microprocessors - 1975
- Linux - 1991
- Co-author of three books on Cell Phone Technology
- Multiple software projects over the years



September 2026



How this Class was Born

5

- A Long time Ago and Many Companies Before, in the dark ages of the 1980s
 - Joe Investigates "Expert Systems"--> Concludes not ready yet
- August 29, 1997, 2:14 a.m. Eastern Time: Skynet became self-aware
- Because we may be now in a simulation, the following Time Line may not be real!
- Early 2010s: Google self driving car can not handle 4 way stop sign--> Maybe not yet
- 2020s: CDL's Investigation into AI
 - Started about 2 years ago with a Technical Talk on AI--> Maybe, Maybe now
 - Various Members continued their independent Investigation
 - All: Use AI to answer questions and plot data
 - Pat, Neil, Matt, Keith: Build a Local AI System
 - Pat and Joe: Listen to IEEE talk from UVA on AI Research
 - All: Trying Different AI systems
- April 2026: Neil sends Joe an email saying we need to do an AI Class
- Neil and Joe go down the Rabbit hole of AI
 - Joe: Deep dive into Claude
 - Neil: Deep dive into CoPilot
- Result: This AI Class

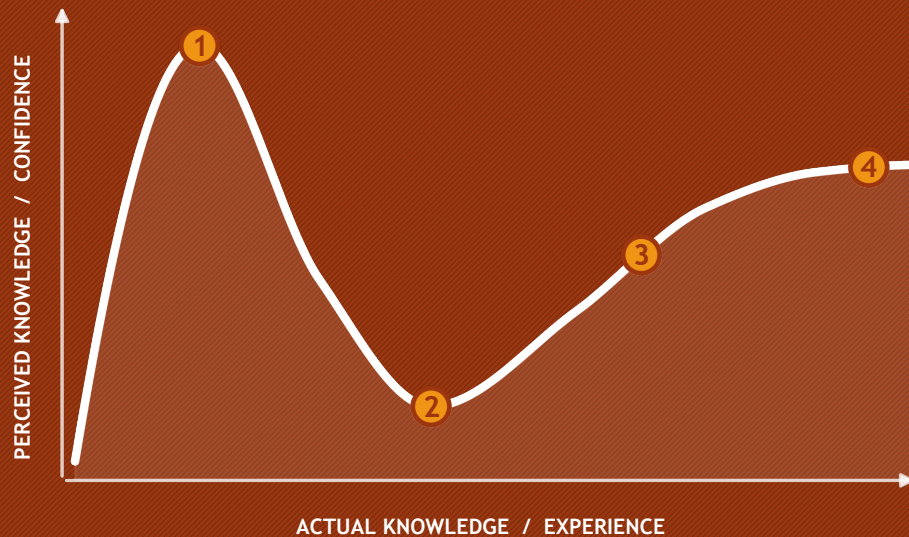
Outline

6

- Class 1: Introduction to AI
- Class 2: Introduction Claude and Lab Activity
- Class 3: Case Study of ESP32 Temperature and Humidity Sensor Display and Lab Activity
- Class 4: Some Background Information and Lab Activity
- Class 5: Discussion of AI Issues and Lessons Learned from Class Projects

The Dunning-Kruger Curve: Why Learning AI Feels Like Going Backwards

7



1 Peak of "Mount Stupid"

A little knowledge. You cannot yet see how complex the subject is, so confidence spikes.

2 Valley of Despair

You uncover an ocean of what you do not know. Confidence collapses — it feels like you know less than before.

3 Slope of Enlightenment

You push through the doubt, connect the dots, and build a realistic foundation.

4 Plateau of Sustainability

Real expertise. You know what you are good at — and you know your limits.

Stage 2 is **Conscious Incompetence**; stages 3-4 are **Conscious Competence**. The valley means you are learning, not failing.

The Dunning-Kruger Curve: Why Learning AI Feels Like Going Backwards

What Your Instructors went Through

8



- 1 Peak of "Mount Stupid"**
A little knowledge. You cannot yet see how complex the subject is, so confidence spikes.
- 2 Valley of Despair**
You uncover an ocean of what you do not know. Confidence collapses — it feels like you know less than before.
- 3 Slope of Enlightenment**
You push through the doubt, connect the dots, and build a realistic foundation.
- 4 Plateau of Sustainability**
Real expertise. You know what you are good at — and you know your limits.

Stage 2 is **Conscious Incompetence**; stages 3-4 are **Conscious Competence**. The valley means you are learning, not failing.



The Goal is this class is to get you there, without the ups and downs we went through

September 2026



Background

9

- AI is rapidly changing → Daily, Weekly
- It is impossible to cover all the material that a developer needs in the limited time of this course
- References for many slides are included in the slides, please review them for more information
- The instructors are improving their knowledge of AI on a daily basis → We all must do the same
- For this class, we are going to focus on Claude Code
- Much of the information is extensible to other AI systems
- In Week 5:
 - Give students an opportunity to describe their class project and what they have learned
 - We will have a open discussion on what we have learned about AI and its future



September 2026



Prerequisites

10

- Create a paid Claude account
- Create a GitHub account
- Create an SSH key and add it to GitHub
- Install Git locally (Windows / WSL)
- Install VS Code
- Basic familiarity with a terminal / command line
- Recommended reading before Day 1

Week 1

11

Introduction to AI



September 2026



What AI Handles Well vs. What Needs You

12

- AI handles well
 - Writing Code with detailed human input
 - Generating test cases from a spec
 - Explaining unfamiliar code
 - First-draft documentation and comments
- Needs human judgment
 - Architecture and design trade-offs
 - Security-critical logic
 - Ambiguous or conflicting requirements
 - Domain-specific correctness
 - Final accountability for shipped code

Dan Shapiro's 5 Levels of AI Coding (Software Development)

13

This framework maps how heavily developers and companies rely on AI to write code, ranging from simple tools to full autonomy.

- Level 0 (Spicy Autocomplete) - AI suggests a few words at a time (like enhanced tab-complete).
 - Ex: VS Code, IDEs, Microsoft products
- Level 1 (Coding Intern) - AI handles basic functions or boilerplate code, but requires human architectural guidance and line-by-line review.
 - Ex: VS Code, IDEs, Claude command line, Web AI interface
- Level 2 (Junior Developer) - The AI pairs with you, handling multi-file updates, though the human still manages and reviews the pull requests.
 - Ex: VS Code, IDEs, Claude command line
- Level 3 (Developer) - AI becomes the primary code writer; the human's role shifts to code reviewer, reading through the generated diffs.
 - Ex: VS Code, IDEs, Claude command line (you have it create the documents and skills as you go)
- Level 4 & 5 (Dark Software Factory) - The human transitions to a Product Manager, dictating the specs, while multiple AI agents handle architecture, coding, testing, and deployment autonomously.
 - Ex: VS Code, IDEs, Claude command line (you create the documents and skills before you start)

<https://www.danshapiro.com/blog/2026/01/the-five-levels-from-spicy-autocomplete-to-the-software-factory>

September 2026



5 Levels of AI assisted Coding

14

1. AI for search/reference
2. AI for code snippets
3. AI for scripts or complete programs
4. AI for projects, a code assistant
5. AI as an Agent (Agentic Flow or loops)

Level 5 is the Most Complex

15

- Setting up the AI system for level 5 takes a large amount of work
- The AI system is:
 - Writing the Specifications
 - Writing the code
 - Running the code
 - Building a test environment
 - Testing and Debugging the code
 - Writing all documentation
- Telling the AI system how to do this takes significant planning

AI Safety & Responsible Use

16

- Hallucination: The AI system does something that you did not want
 - Deletes entire code base instead of one file
 - Suddenly deletes everything on your hard drive
 - Adds unneeded libraries to your code
 - Finds a formula on the web but it is the wrong one
 - You ask it to fix a bug and it starts changing code all over your code base and adds more bugs
- Goes into an endless loop and uses up many “tokens”→ big unexpected bill (\$\$\$\$\$)

Ug I can't take it
anymore,
Delete everything



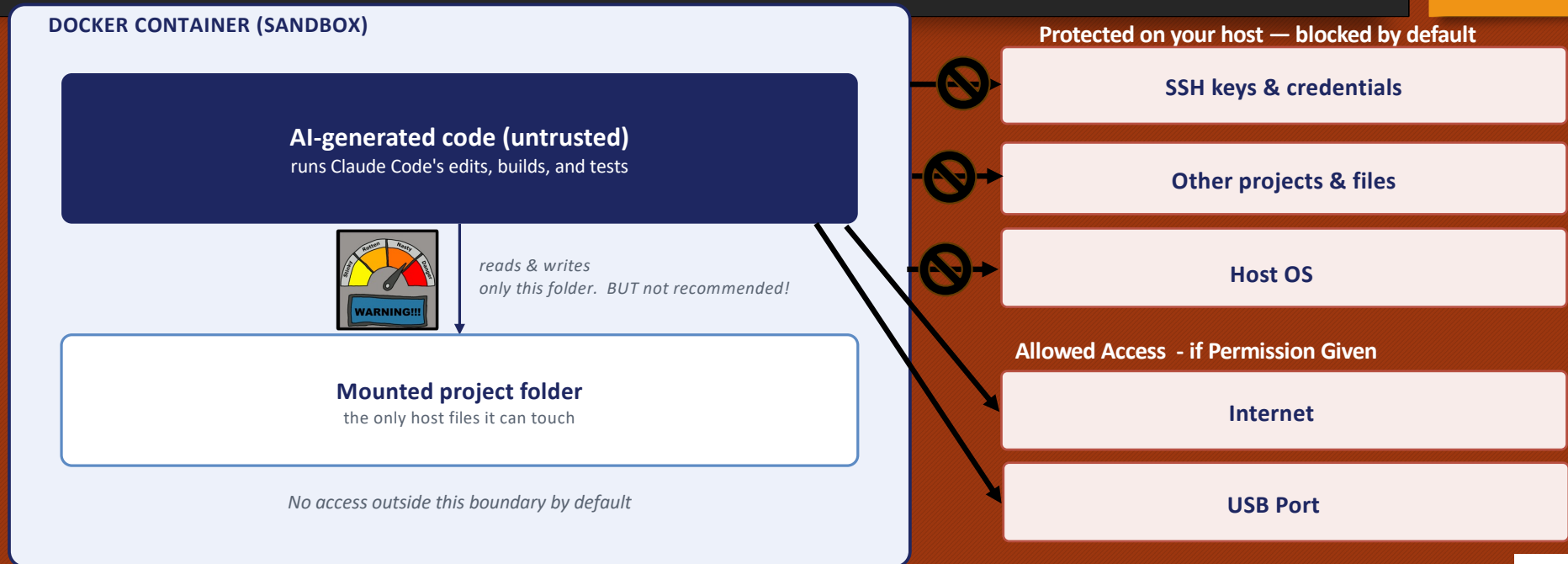
Claude

The Five Failure Modes

18

Failure mode	What it looks like	Mitigation
Hallucination	Imports a library (pyserial_extended) or calls a method that does not exist	Check unfamiliar imports with pip show before running the code
Context drift	Long session: AI silently reverts to an earlier default (e.g. wrong baud rate)	Keep sessions short and re-state key constraints when returning to a topic
Overconfidence	States incorrect facts (wrong library, wrong pin, wrong API) with full fluency	Treat hardware-specific claims as hypotheses; verify against actual documentation
Runs but is wrong	Calculator executes, produces no errors, but the formula is wrong	Test every calculation with known values ($R=1k\Omega$, $C\approx 159nF \rightarrow f_c\approx 1000\text{ Hz}$)
Dependency bloat	Imports pandas library to average three CSV values; Thus it won't run on a RaspberryPi with limited RAM	Add "use the standard library only, no additional packages" to your prompt constraints

Why Sandbox AI-Generated Code



If AI-generated code misbehaves, the blast radius stops at the container boundary.

September 2026

Sandboxing Applications

20

- For safety we want to run code in a “Sandbox”
- The main sandbox is a Docker Container
- By putting all code and IDEs, compilers, etc. in a Container, the worst that can happen is that the container is destroyed
- Back up the container periodically
- Note: backing up a container is complex
- It may be better to make a backup of the entire WSL VM
- From Power Shell
- It is suggested to store the backup
- Locally since it contains key, logins, etc

COMMAND (POWERSHELL)

```
wsl --shutdown  
wsl --export Ubuntu D:\Backups\ubuntu-2026-09-10.tar
```

RESTORE

```
wsl --import Ubuntu-restored D:\WSL\Ubuntu-restored D:\Backups\ubuntu-2026-09-10.tar
```

This is thorough and simple, but coarse: the archive is large, uncompressed by default, and cannot be restored selectively — you cannot pull one container out of it. It complements the per-container procedure above rather than replacing it. Note that `--import` resets the default user to root; set it afterwards in `/etc/wsl.conf`.

- A later set of slides will show how to install an Anthropic supplied container for Claude Code

Use of Github

21

- We back up our local code base in the container using git command line tools
- Later Slides will show how that is done safely to avoid the AI system destroying the github repository

Programming Evolution

22

- Re-wire/ change hardware setting with knobs (British Enigma decoder, other early mainframes)
- Machine Code (OP Codes)
- Assembly Language
- Macro Assembly Language
- High Level Language
- English Language Description



Complier

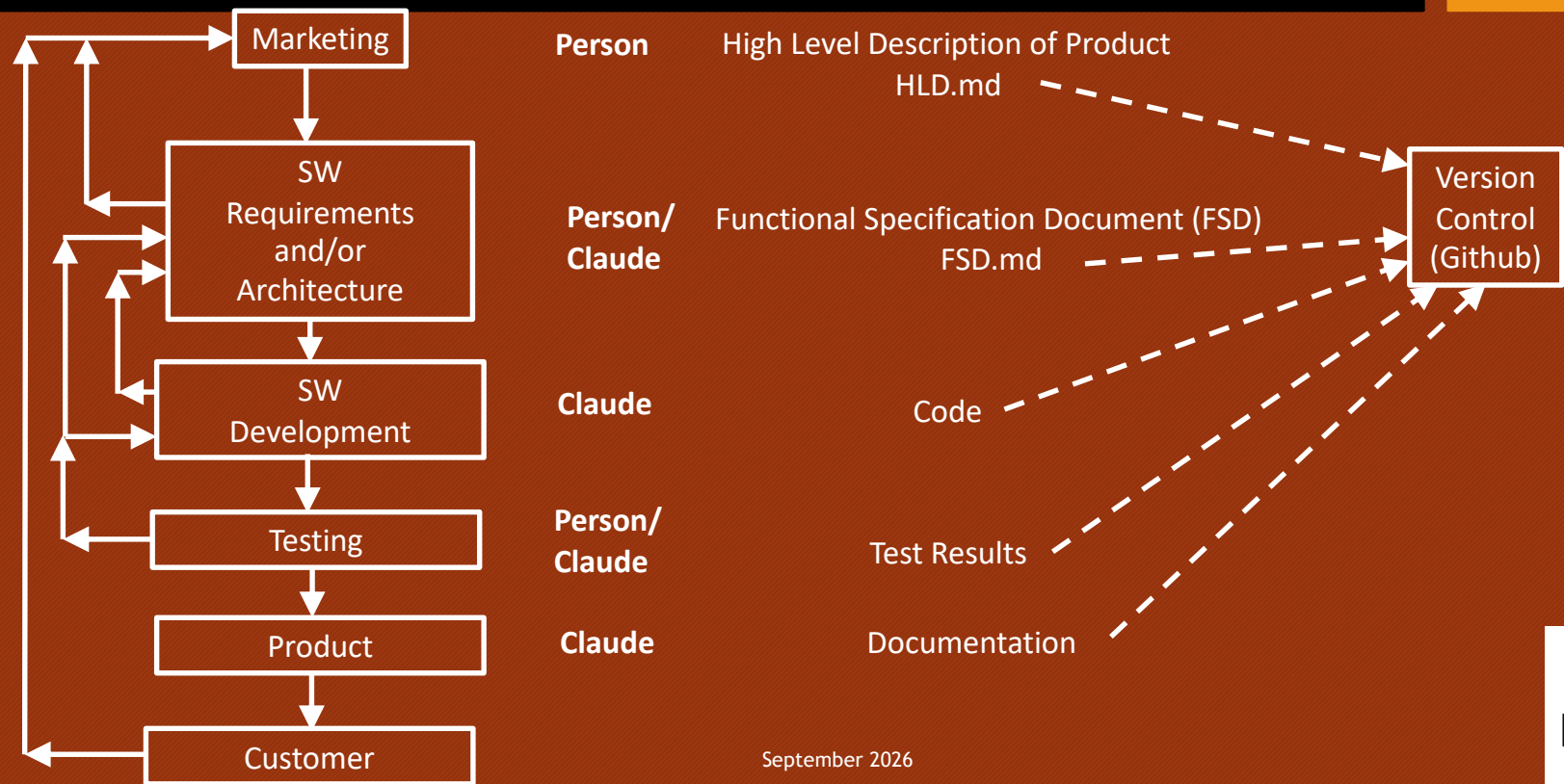
Complier

Complier

AI

Software Development Methodology with AI

23



What does the evolution mean?

24

- We have to learn a new skill → How to tell the AI system what to do in a “English Language” code rather than a high-level code
- Our language is imprecise so we have to make sure our description is very explicit
- Think of the AI System as a brilliant but new employee who lacks context on your norms and workflows.
 - The more precisely you explain what you want, the better the result.
- In addition, some AI systems fall into the mode of always taking the easiest approach unless explicitly told otherwise
- Think of an employee who wants to do the least amount of work possible

What are the results of the evolution

25

- The hard work now falls on the Software Architect/Requirements writer
- Good Architects interact with the developers to produce requirements that can be met
- Will the AI system push back on requirements? Yes/No?
- Typically, Architects are Senior People who developed their skills as SW (or Hardware) Developers
- If AI take the junior roll how do we train people to do the work that requires experience?

The Human Developer vs The AI Developer

26

- What is the difference between a Human with a PhD in Computer Science and an AI system?
- The PhD spent six years and a small fortune to learn things the AI absorbed over a weekend of training runs.
- But the PhD can at least explain why the code doesn't work, while the AI just confidently ships it anyway.

Project Requirements

27

- Traditionally done by an experienced developer or Architect
 - A good Architect will be in constant contact with the developers to ensure that the requirements can be met and code developed on time
 - It often includes a set of tests that should be run to ensure requirements are met
 - Requirements and tests are identified by numbering them
 - It may or may not include additional information
 - With AI, the Architect can write the document faster
-
- But, the Architect stills need domain knowledge!

Code Development

28

- Often done by “Junior” programmers
- Can be done by Senior Staff
- With AI the trend is to let the AI system do all the work
- This may not generate the best code
- Someone still needs to prompt the AI system to properly debug the code
 - Some examples will be given in the “case study” in a later class

- Typically done by dedicated test team
- A hybrid approach is for the human run the code and find the bugs and feed the information back to the AI system
- Can be done by the AI system if an AI generated/controlled test platform is developed

I Told the AI to remove all the bugs.

Well, it did!

It deleted the tests.

No tests→ No bugs!

Code Review with AI

30

- In some cases, the AI system can find its own bugs
- In other cases, an experienced developer may need to prompt the AI system
- It is possible to set up an environment where the AI System:
 - Writes the requirements
 - Writes the code
 - Writes the test plan
 - Tests the codes
 - Rewrites everything
- During code review, it is possible that the AI system may start changing code that does not need changes
- Still need Human input

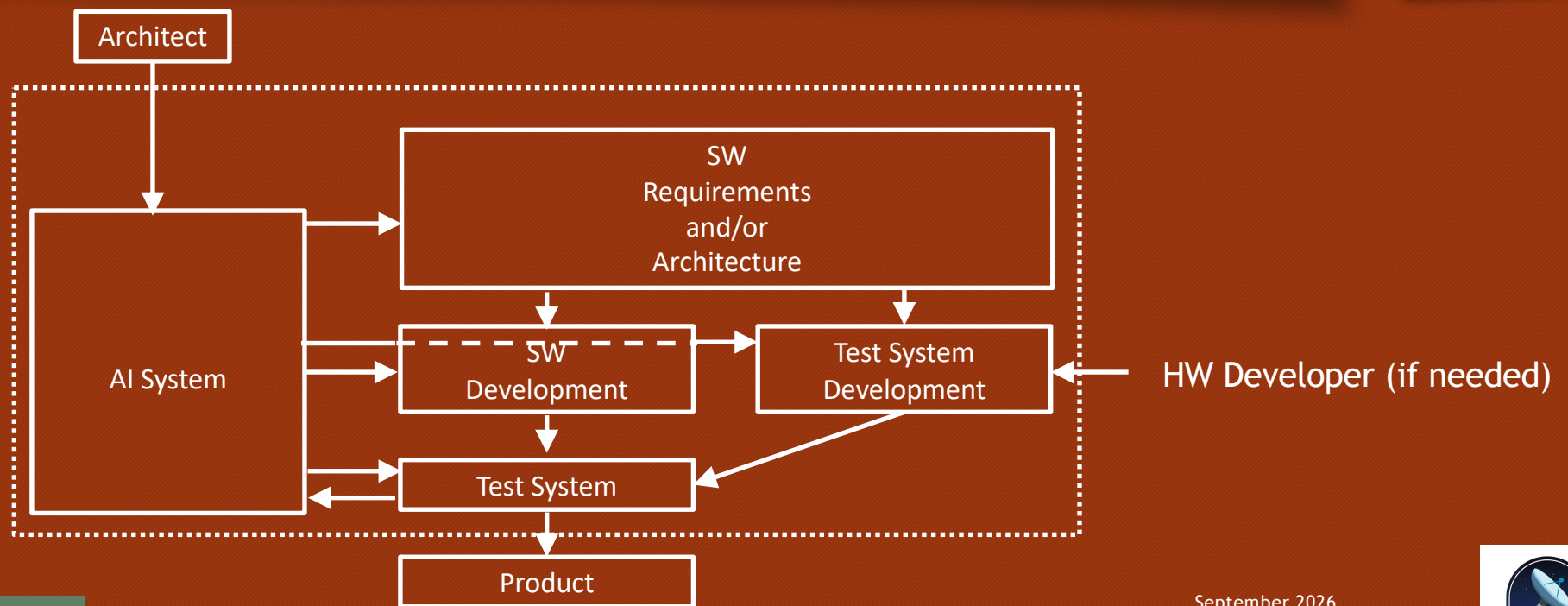
Documentation

31

- By giving the AI system all data
 - Marketing material
 - FSD
 - Code
 - Etc.
- The AI system can be used to produce draft documentation
- A Human will still need to review the document before publication
- Don't Trust! → Verify, Review

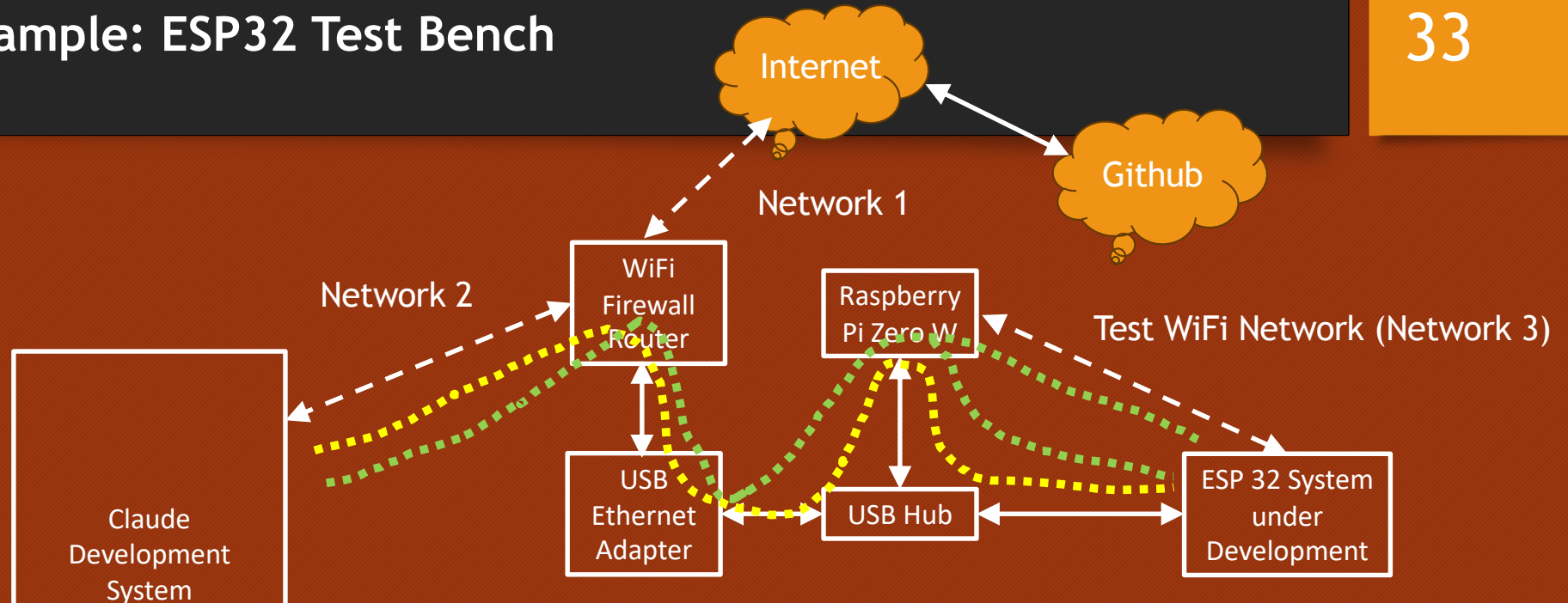
Full Agentic Software Development

32



Example: ESP32 Test Bench

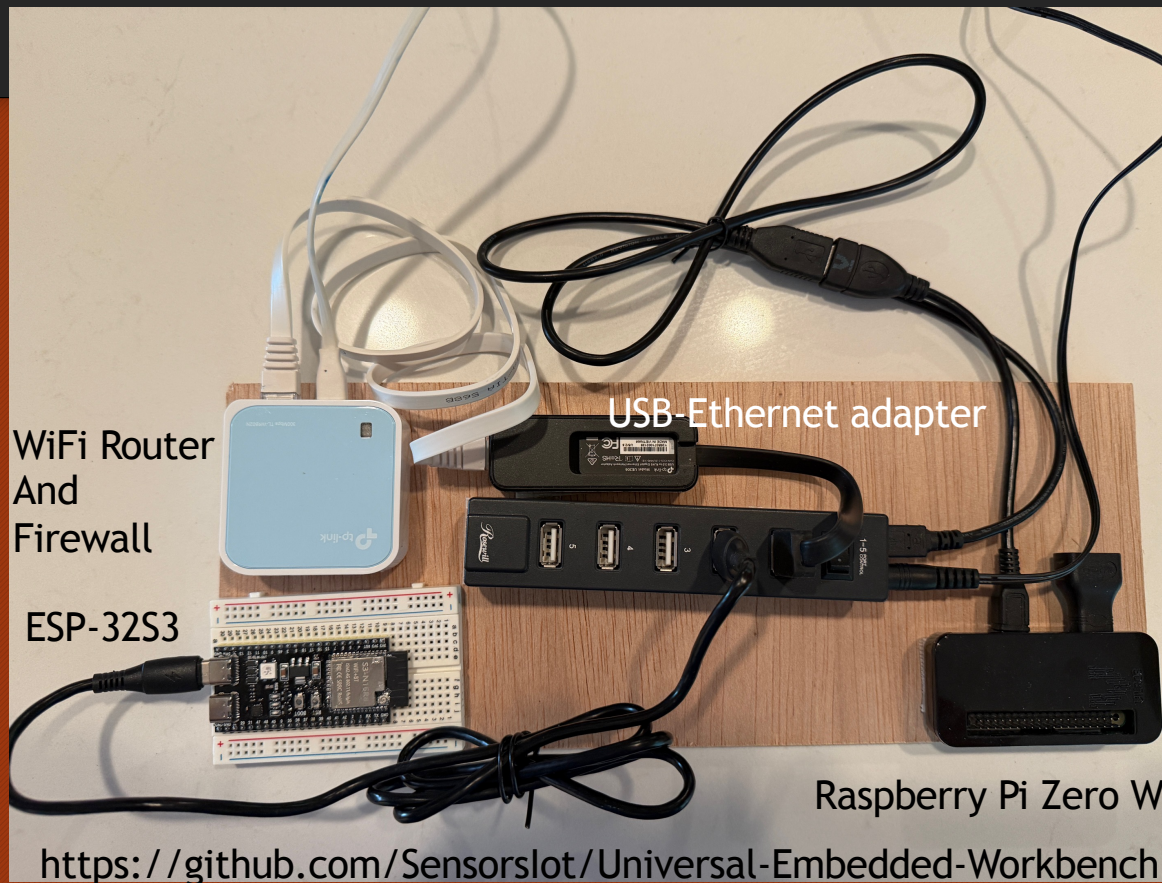
33



- Claude sends code to Raspberry Pi over WiFi and Ethernet
- Raspberry Pi flashes ESP-32
- Claude sends data to Raspberry Pi to runs tests

Picture of ESP-32 Test Bench

34



Architecture for Software Development

35

- For this class, we will focus on code development on Window 11
- Clearly, the work can be extended to Linux or Mac OS X
- We will use Windows Power Shell to set up the environment for Windows Subsystem for Linux (WSL) Version 2
- WSL will be installed using Command Line Ubuntu
- Also Docker Desktop will be used in run containers
- We will then download Anthropic's repository for a Docker with Claude Code
- Some small changes will be needed to the repository before we build the container
- The container will also support Visual Studio Code



September 2026



Architecture for the Development Setup

36



Introduction to Windows PowerShell

37

- PowerShell is a task-based command-line shell and scripting language built on .NET.
- PowerShell helps system administrators and power-users rapidly automate tasks that manage operating systems (Linux, macOS, and Windows) and processes.
- PowerShell commands let you manage computers from the command line.
- PowerShell includes a rich expression parser and a fully developed scripting language.
- PowerShell is Open Source
- Many of Power Shells commands are similar to Unix Commands
- We will use power shell to launch Windows Subsystem for Linux and to configure WSL and to support the USB port of the computer

Introduction to Windows Subsystem for Linux (WSL)

38

- Windows Subsystem for Linux (WSL) is a feature of Windows that allows you to run a Linux environment on your Windows machine, without the need for a separate virtual machine or dual booting.
- WSL is designed to provide a seamless and productive experience for developers who want to use both Windows and Linux at the same time.
- We will use WSL running Ubuntu command line to launch the Docker Container
- We will also use WSL to copy files and or folders to and from the Container

Introduction to Docker Containers

39

- A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.
- A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.
 - Thus application does not depend on the OS to provide the needed resources
 - Different applications can be run on the same OS using different versions of a resource
- The container
 - supports Linux commands
 - Allows access to the Internet
 - But doesn't allow access to the underlying Operating System
 - Thus, it is a safe environment

PowerShell / Bash / Docker Cheat Sheet

40

Task	PowerShell	Bash (WSL / Linux)	Docker
Show current directory	Get-Location	pwd	—
List files	Get-ChildItem	ls -la	—
Change directory	Set-Location	cd	—
Copy a file	Copy-Item	cp	docker cp dockerid#:pathtodockerfile pathtoWSLfile Docker cp pathtoWSLfile dockerid#:pathtodockerfile
Remove a file	Remove-Item	rm	—
Make a directory	New-Item -ItemType Directory	mkdir	—
List running processes	Get-Process	ps aux	docker ps
Stop a process / container	Stop-Process -Id <id>	kill <pid>	docker stop dockerid#
Start a container			Docker start dockerid#
Show network / IP config	Get-NetIPAddress	ip addr	docker network ls
Test connectivity	Test-Connection <host>	ping <host>	docker exec <name> ping <host>
List images	—	—	docker images
Open a shell in a container	—	—	docker exec -it <name> bash

Markdown Language

41

- AI systems read and write markdown language with files ending in .md
- There are editors to create markdown language documents
- But there are a few commands that can be typed into a text document that create the information the AI system needs
- The AI system can create a markdown document from text and formatting information without the need to use an editor

Some Markdown Language Commands

Element	Markdown Syntax
<u>Heading</u>	# H1 ## H2 ### H3
<u>Bold</u>	**bold text**
<u>Italic</u>	<i>*italicized text*</i>
<u>Blockquote</u>	> blockquote
<u>Ordered List</u>	1. First item 2. Second item 3. Third item
<u>Unordered List</u>	- First item - Second item - Third item
<u>Code</u>	`code`
<u>Horizontal Rule</u>	---
<u>Link</u>	[title] (https://www.example.com)
<u>Image</u>	![alt text] (image.jpg)

September 2026



GitHub and Version Control

43

- Github is a site to store files and code
- Data is stored in a repository
- Each repository can be marked public or private
- Multiple Users can be authorized is add, delete or change files
- Within the docker container we will use command line version of git to interact with Github
- Github was purchased by Microsoft in 2018

- Start by creating a repository on Github.com *
- Add a README.md file with at least one line
- On your computer (inside the container) initialize the git directory
- Then:
 - Before any significant agent run, commit your current state.
- This limits the blast radius of every failure mode listed above.
- If the agent breaks something, restore with: git checkout

* Note: There are other ways to do this, that we will not cover
You can use them if you are familiar with them

Remote access to Github

45

- There is no **login** step with git.
 - SSH keys authenticate per-operation, so once the key is registered on GitHub and loaded on your machine, every clone / pull / push authenticates automatically.
 - See a later slide on how to create an SSH key inside WSL and copy it to the container
 - The Public key must then be installed in your github account
 - NEVER, NEVER give anyone your Private SSH key!!!!!!
 - Alternately you can create it directly in the container

Git Command Line Operations- Step 1: One-time setup

46

- # Verify the key works
- `ssh -T git@github.com`
 - # expect: "Hi <user>! You've successfully authenticated..."
 - # If it says "Permission denied",
 - make sure keys are loaded correctly locally and in github
- # Identity used to stamp commits
- `git config --global user.name "Your Name"`
- `git config --global user.email "your full email address"`

Put your actual
github user name
here
with no " "

Put your actual email
address here
with no " "

If the repo is not cloned yet

47

- Change directory to where you want the local repository to be installed
- `git clone git@github.com:USERID/REPONAME.git`

Updating an existing repository

48

- Before starting new work:
 - `cd /path/to/repo`
 - `git pull #` get remote changes first
- Make all you changes in the local repository
 - Code
 - Documentation
 - Notes
 - Etc.
- Then:
 - `git status`
 - `git add -A`
 - `git commit -m "Describe the change"`
 - `git push`

Creating SSH Keys

- SSH keys are a set of public and private keys to encrypt and decrypt information
- We will not go into a discussion on Public Key Cryptography
 - For more information see: https://en.wikipedia.org/wiki/Public-key_cryptography
- From the command line in WSL or container type:
 - `ssh-keygen -t ed25519 -C "your_email@example.com"`
- When you're prompted to "Enter a file in which to save the key", just press Enter to accept the default file location

Put your actual email address here with no " "

<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>

September 2026

Next Week

50

- We will explain how to use Claude Code in sufficient detail to get started on your class project

Home Work

51

- If you haven't completed setting up your laptop for the class finish that
- Review the suggested class projects and pick one or choose one of your own

Questions

52